2022

SISTEMAS EN CHIP

PRACTICA 08 “CRONOMETRO DE 60 SEGUNDOS”

FRANCISCO UZIEL CORDOVA PICHARDO

Objetivo

Al término de la sesión, los integrantes del equipo contaran con la habilidad de realizar un teclado matricial.

Introducción Teórica

Un teclado matricial es un tipo de teclado que utiliza una disposición matricial de interruptores para detectar la pulsación de teclas. En lugar de tener un interruptor individual para cada tecla, los interruptores de un teclado matricial están conectados en una matriz de filas y columnas. Al presionar una tecla, se cierra un circuito en una determinada posición de la matriz, permitiendo al sistema detectar qué tecla fue pulsada.

El ATmega8535 es un microcontrolador de 8 bits basado en la arquitectura AVR de Atmel. Es capaz de ejecutar programas almacenados en su memoria flash y ofrece una variedad de periféricos, como entradas/salidas digitales y analógicas, timers y comunicación serie. Es comúnmente utilizado en proyectos de robótica, automatización y control de dispositivos. Se puede utilizar junto con un teclado matricial para recibir y procesar la señal de las teclas pulsadas, y luego utilizar el microcontrolador para controlar un dispositivo o tomar una decisión en función de la tecla pulsada.

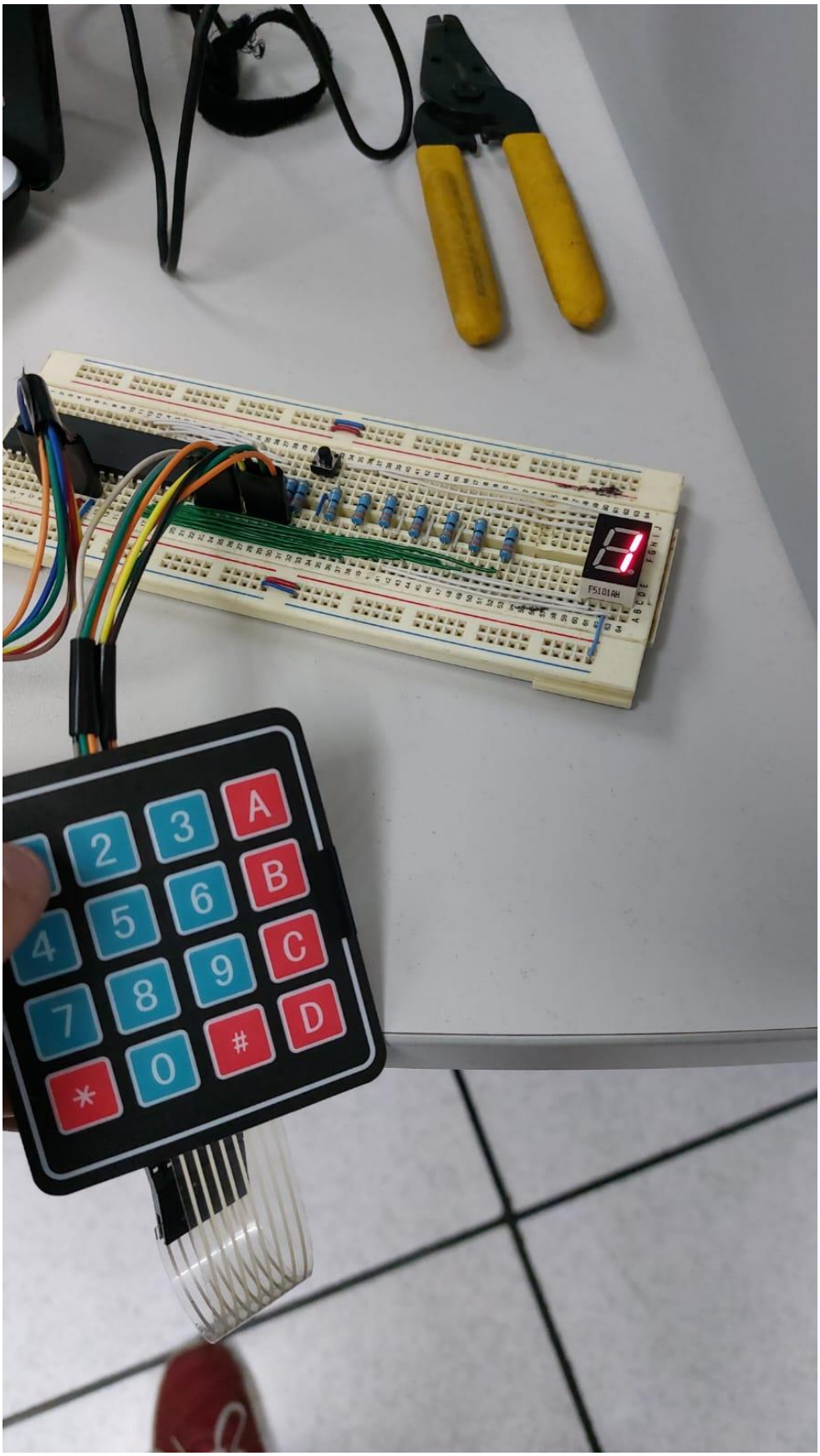
Desarrollo Experimental

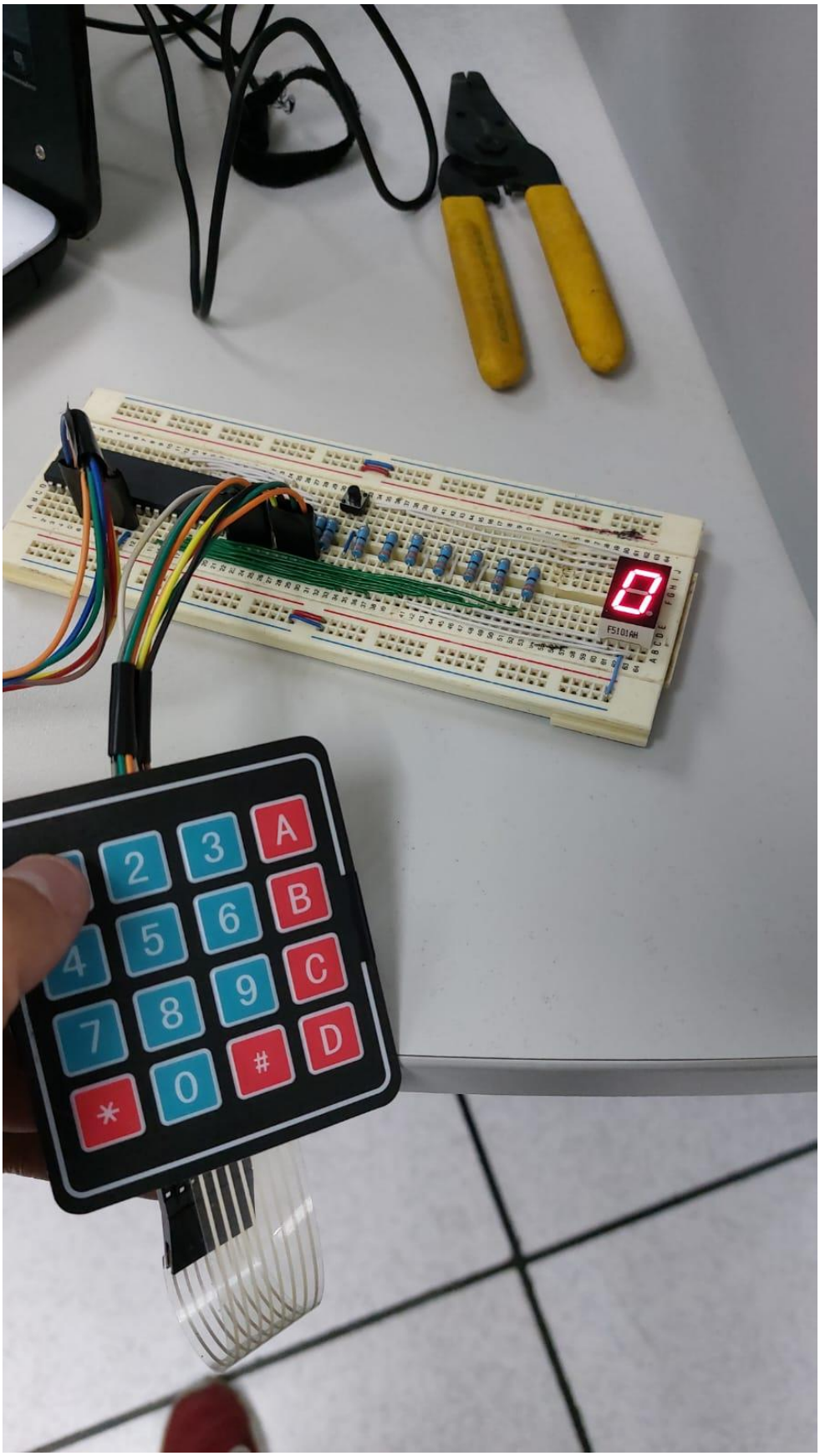
1.- Diseño de un teclado matricial de 3*3, con despliegue a display a 7 segmentos. Los pines C0, C1 y C2 serán los pines de salida

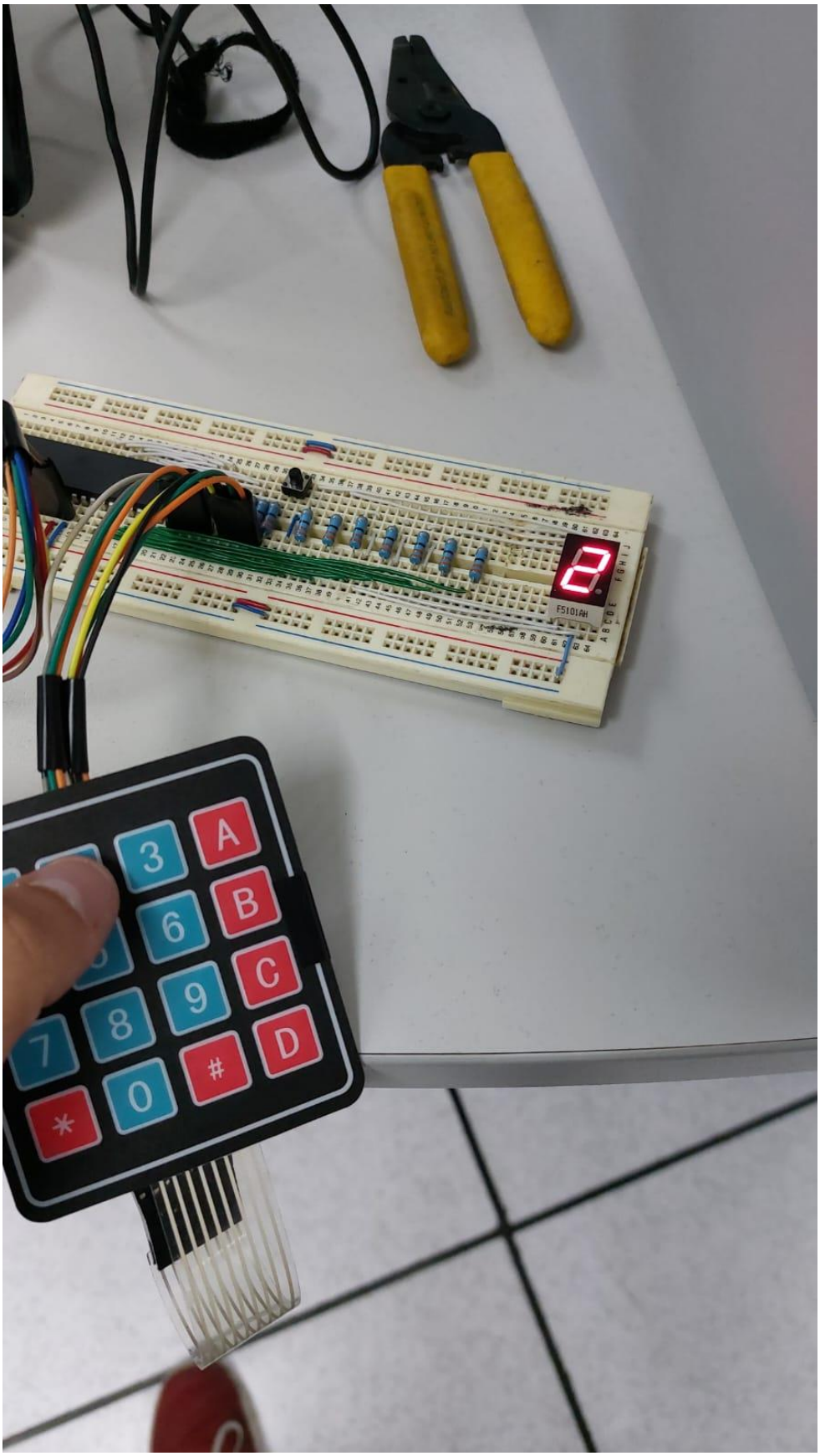
por donde se enviarán los códigos de “ascenso”, los pines C3, C4 y C5 serán los pines de entrada donde se leerán los botones

presionados.

Fotografías del circuito funcionando


2
3
A
6
B
5
4
9
C
8
7
#
D
0
*
F5101AH


2
3
A
B
5
6
4
C
8
9
7
D
#
0
*


3
A
6
B
9
C
8
7
D
#
0
*
2
F5101AH

Código del programa

```
/*****************************************************
This program was created by the CodeWizardAVR V3.32
Automatic Program Generator
© Copyright 1998-2017 Pavel Haiduc, HP InfoTech s.r.l.
http://www.hpinfotech.com

Project :
Version :
Date    : 10/11/2022
Author  :
Company :
Comments:


Chip type               : ATmega8535
Program type            : Application
AVR Core Clock frequency: 1.000000 MHz
Memory model            : Small
External RAM size       : 0
Data Stack size         : 128
*****************************************************/

#include <mega8535.h>
// Declare your global variables here
unsigned char tecla,lectura;
const char tablaHex
[16]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,0x77,0x7c,0x39,0x5e,
0x79,0x71};
void main(void)
{
// Declare your local variables here

// Input/Output Ports initialization
// Port A initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRA=(1<<DDA7) | (1<<DDA6) | (1<<DDA5) | (1<<DDA4) | (1<<DDA3) | (1<<DDA2) |
(1<<DDA1) | (1<<DDA0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) |
(0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);

// Port B initialization
```

```
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRB=(1<<DDB7) | (1<<DDB6) | (1<<DDB5) | (1<<DDB4) | (1<<DDB3) | (1<<DDB2) |
(1<<DDB1) | (1<<DDB0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) |
(0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);

// Port C initialization
// Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (1<<DDC3) | (1<<DDC2) |
(1<<DDC1) | (1<<DDC0);
// State: Bit7=P Bit6=P Bit5=P Bit4=P Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTC=(1<<PORTC7) | (1<<PORTC6) | (1<<PORTC5) | (1<<PORTC4) | (0<<PORTC3) |
(0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

// Port D initialization
// Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out
Bit0=Out
DDRD=(1<<DDD7) | (1<<DDD6) | (1<<DDD5) | (1<<DDD4) | (1<<DDD3) | (1<<DDD2) |
(1<<DDD1) | (1<<DDD0);
// State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) |
(0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

// Timer/Counter 0 initialization
// Clock source: System Clock
// Clock value: Timer 0 Stopped
// Mode: Normal top=0xFF
// OC0 output: Disconnected
TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) |
(0<<CS01) | (0<<CS00);
TCNT0=0x00;
OCR0=0x00;

// Timer/Counter 1 initialization
// Clock source: System Clock
// Clock value: Timer1 Stopped
// Mode: Normal top=0xFFFF
// OC1A output: Disconnected
// OC1B output: Disconnected
// Noise Canceler: Off
// Input Capture on Falling Edge
// Timer1 Overflow Interrupt: Off
```

```
// Input Capture Interrupt: Off
// Compare A Match Interrupt: Off
// Compare B Match Interrupt: Off
TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) |
(0<<WGM10);
TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) |
(0<<CS11) | (0<<CS10);
TCNT1H=0x00;
TCNT1L=0x00;
ICR1H=0x00;
ICR1L=0x00;
OCR1AH=0x00;
OCR1AL=0x00;
OCR1BH=0x00;
OCR1BL=0x00;

// Timer/Counter 2 initialization
// Clock source: System Clock
// Clock value: Timer2 Stopped
// Mode: Normal top=0xFF
// OC2 output: Disconnected
ASSR=0<<AS2;
TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) |
(0<<CS21) | (0<<CS20);
TCNT2=0x00;
OCR2=0x00;

// Timer(s)/Counter(s) Interrupt(s) initialization
TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) |
(0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);

// External Interrupt(s) initialization
// INT0: Off
// INT1: Off
// INT2: Off
MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
MCUCSR=(0<<ISC2);

// USART initialization
// USART disabled
UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) |
(0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

// Analog Comparator initialization
// Analog Comparator: Off
```

```
// The Analog Comparator's positive input is
// connected to the AIN0 pin
// The Analog Comparator's negative input is
// connected to the AIN1 pin
ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) |
(0<<ACIS1) | (0<<ACIS0);
SFIOR=(0<<ACME);

// ADC initialization
// ADC disabled
ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) |
(0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);

// SPI initialization
// SPI disabled
SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) |
(0<<SPR1) | (0<<SPR0);

// TWI initialization
// TWI disabled
TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

while (1)
      {
        PORTC=0b11111110;
        lectura = PINC&0b11110000;
        if(lectura==0b11100000){
            tecla=10;
        }
        if(lectura==0b11010000){
            tecla=11;
        }
        if(lectura==0b10110000){
            tecla=12;
        }
        if(lectura==0b01110000){
            tecla=13;
        }
        //cambiamos la columna
        PORTC=0b11111101;
        lectura = PINC&0b11110000;
        if(lectura==0b11100000){
            tecla=1;
        }
        if(lectura==0b11010000){
```

```
            tecla=4;
        }
        if(lectura==0b10110000){
            tecla=7;
        }
        if(lectura==0b01110000){
            tecla=14;
        }
        //cambiamos la columna
        PORTC=0b11111011;
        lectura = PINC&0b11110000;
        if(lectura==0b11100000){
            tecla=2;
        }
        if(lectura==0b11010000){
            tecla=5;
        }
        if(lectura==0b10110000){
            tecla=8;
        }
        if(lectura==0b01110000){
            tecla=0;
        }
        //cambiamos la columna
        PORTC=0b11110111;
        lectura = PINC&0b11110000;
        if(lectura==0b11100000){
            tecla=3;
        }
        if(lectura==0b11010000){
            tecla=6;
        }
        if(lectura==0b10110000){
            tecla=9;
        }
        if(lectura==0b01110000){
            tecla=15;
        }

        PORTD=tablaHex[tecla];
      }
}
```

Conclusiones

En conclusión, el teclado matricial es una forma eficiente y económica de detectar la pulsación de teclas, ya que utiliza una disposición matricial de interruptores en lugar de tener un interruptor individual para cada tecla. El ATmega8535 es un microcontrolador de 8 bits capaz de ejecutar programas y ofrece una variedad de periféricos que pueden ser utilizados para recibir y procesar la señal de las teclas pulsadas en un teclado matricial. Esto permite al sistema detectar qué tecla fue pulsada y tomar decisiones en consecuencia, lo que es esencial en proyectos de robótica, automatización y control de dispositivos.

Bibliografía

Manual del ATmega8535: https://pdf1.alldatasheet.es/datasheet-pdf/view/164169/ATMEL/ATMEGA8535.html